

A high accuracy surface modeling method based on GPU accelerated multi-grid method

Changqing Yan¹ | Jimin Liu¹ | Gang Zhao² | Chuanfa Chen³ |
Tianxiang Yue⁴

¹Department of Information Engineering,
Shandong University of Science and
Technology, Daizong Avenue, Taishan
District, 271000 Taian, China

²Crop Science Group, Institute of Crop
Science and Resource Conservation (INRES),
University of Bonn, Katzenburgweg 5,
D-53115 Bonn, Germany

³State Key Laboratory of Mining Disaster
Prevention and Control, Shandong
University of Science and Technology,
Qingdao, China

⁴Institute of Geographical Sciences and
Natural Resources Research, Chinese
Academy of Sciences, 11A, Datun Road,
Anwai 100101, Beijing, China

Correspondence

Chuanfa Chen, State Key Laboratory of
Mining Disaster Prevention and Control,
Shandong University of Science and
Technology, 266590 Qingdao, China.
Email: chenxf@leis.ac.cn

Abstract

A high accuracy surface modeling method (HASM) has been developed to provide a solution to many surface modeling problems such as DEM construction, surface estimation and spatial prediction. Although HASM is able to model surfaces with a higher accuracy, its low computing speed limits its popularity in constructing large scale surfaces. Hence, the research described in this article aims to improve the computing efficiency of HASM with a graphic processor unit (GPU) accelerated multi-grid method (HASM-GMG). HASM-GMG was tested with two types of surfaces: a Gauss synthetic surface and a real-world example. Results indicate that HASM-GMG can gain significant speedups compared with CPU-based HASM without acceleration on GPU. Moreover, both the accuracy and speed of HASM-GMG are superior to the classical interpolation methods including Kriging, Spline and IDW.

KEYWORDS

computing speed, GPU, HASM, multi-grid method, surface modeling

1 | INTRODUCTION

Surface modeling is a procedure used to simulate a surface through a sample scattered point-form data set or a line-form data set (Childs, 2004; Yue, 2011). Surface modeling has widespread applications in many fields, especially in earth sciences such as DEM construction, surface approximation and spatial prediction, and change detection (Yue, 2011).

All applications are inevitably subject to errors due to many factors involved in the modeling process, of which the employed numerical modeling methods play an important role (Fisher & Tate, 2006). To give a solution to the error problem caused by the numerical modeling methods, a novel surface modeling method, termed high accuracy surface modeling (HASM), was recently developed based on the fundamental theorem of surfaces in differential geometry (Yue, 2011). Previous studies indicated that HASM is superior to the classical interpolation methods including Kriging, Spline, and Inverse Distance Weight (IDW) in many applications including DEM modeling, ecological diversity modeling and spatial change trend detection (Yue, 2011).

Despite its high accuracy, HASM suffers from low computing efficiency due to the solution of a large linear system (Yue, Du, Song, & Gong, 2007). To shorten the computing time, both direct and iterative algorithms have been employed to solve the system of HASM (Yue, 2011). The direct algorithms include the Gaussian elimination algorithm and the square-root algorithm, and the iterative ones include the Gauss-Seidel (GS) algorithm, the preconditioned Gauss-Seidel (PGS) algorithm and the preconditioned conjugate gradient (PCG) algorithm. A series of numerical tests demonstrated that the PCG-based iterative algorithm (HASM-PCG) is much faster than other algorithms of HASM in terms of simulation accuracy and computational speed. Additionally, multi-grid was introduced to solve HASM and obtained an comparable computing efficiency to that of PCG (Yue et al., 2013). Yet the computing speed of both PCG and multi-grid are still not fast enough for many practical uses and large-scale surface modeling problems (Yue et al., 2013). Moreover, due to the versatility of multi-grid, it has great computing efficiency improvement potential (Briggs & McCormick, 2000). Multi-grid methods can be designed to be composed of completely data-parallel operations, which deal with many data elements with the same thread, and these operations can greatly benefit from parallel computing technology, which divides large problems into smaller ones solved at the same time (Grossauer & Thoman, 2008). More frequently the method is implemented in a serial computing approach and the suitability of using parallel computing to accelerate the algorithm has not been studied.

Designing parallel algorithms and harnessing more computing power from advanced hardware is becoming another increasingly important way to shorten computing time. It has been proved that the utilization of a many-core processor, computing clusters, and distributed computing can significantly speed up many demanding computing applications (Hackbusch, 1985; Helfenstein & Koko, 2012; Zhao et al., 2013; Zhao, Bryan, King, Song, & Yu, 2012). Compared with computing clusters and grid computing (Zhao et al., 2013), the general purpose graphic processor unit (GPGPU) (Bryan, 2013), equipped with many cores, is advantageous for its energy efficiency, low cost, wider availability and high performance-to-price ratio, and appropriate to be used for intensive computing acceleration.

Many real-world applications in geographic information science (GIS) benefit from many-core GPU (Chao, Chongjun, Zhuo, Xiaojing, & Hantao, 2011; Feng et al., 2015; Zhao, Padmanabhan, & Wang, 2013). For interpolation, Beutel, Mülhave, and Agarwal (2010) presented a GPU accelerated nearest neighbor interpolation algorithm for DEM and obtained a 10 fold speedup over standard linear interpolation. Pan and Manocha (2011) presented a GPU based k-nearest neighbor algorithm and obtained a 40x acceleration rate. De Ravé, Jiménez-Hornero, Ariza-Villaverde, and Gómez-López (2014) used GPU to accelerate the Kriging algorithm. Numerous other GIS algorithms or models have also been accelerated by GPUs. For example, geospatial overlay computation (McKenney, De Luna, Hill, & Lowell, 2011), map projection (Zhao, Cheng, Dong, Fang, & Li, 2011), slope computation and map matching algorithm (Stojanovic & Stojanovic, 2013) were accelerated on GPU. A hydrology model (Qin & Zhan, 2012), and a drainage network computation (Ortega & Rueda, 2010) were also accelerated. To meet the increasing computing requirements in GIS, Tabik, Villegas, Zapata, & Romero (2012) employed GPU to develop a fast GIS tool. Zhang and You (2012) designed a CUDA-GIS software based on GPU and obtained 20x to 40x speedups.

Therefore, HASM may also benefit from the utilization of GPU. Namely, if the flexibilities of multi-grid could be utilized to design parallel algorithms running on many-core GPUs and employed to solve HASM, it could be expected to exhibit significant performance improvement. The objective of the research described in this article was to first develop a GPGPU accelerated multi-grid method to solve HASM (HASM-GMG) and then compare its performance with the classical algorithms of HASM.

The remainder of this article is organized as follows. Section 2 first gives a brief introduction to HASM, multi-grid and GPU high performance computing, and then develops the algorithm of HASM-GMG. Section 3 compares the performance of HASM-GMG with the other algorithms of HASM. In Section 4 and the last section, some discussion and conclusion are given.

2 | GPU ACCELERATED MULTI-GRID BASED HASM SOLUTION

2.1 | The principle of HASM

HASM is a novel surface modeling method, which aims to give a solution to the error problems of surface modeling (Yue, 2011). It is developed based on the fundamental theorem of surfaces, which uniquely determines a surface by

the first and the second fundamental coefficients. According to this theorem, a surface can be represented by a group of partial differential equations (PDE):

$$\begin{cases} f_{xx} = \Gamma_{11}^1 \cdot f_x + \Gamma_{11}^2 \cdot f_y + L \cdot (E \cdot G - F^2)^{-1/2} \\ f_{xy} = \Gamma_{12}^1 \cdot f_x + \Gamma_{12}^2 \cdot f_y + M \cdot (E \cdot G - F^2)^{-1/2} \\ f_{yy} = \Gamma_{22}^1 \cdot f_x + \Gamma_{22}^2 \cdot f_y + N \cdot (E \cdot G - F^2)^{-1/2} \end{cases} \quad (1)$$

where $\Gamma_{11}^1, \Gamma_{11}^2, \Gamma_{12}^1, \Gamma_{12}^2, \Gamma_{22}^1$, and Γ_{22}^2 are coefficients of connection, (L, M, N) , and (E, F, G) are respectively the first and the second fundamental coefficients, and (f_x, f_y) and (f_{xx}, f_{xy}, f_{yy}) are the first order derivatives and second order derivatives with respect to x and y , respectively.

To solve this PDE group, the finite difference or finite element strategy can be used. The finite element method is generally employed on an unstructured grid, while the finite difference method is employed on structured grids. Hence, finite difference scheme was adopted to discretize it on structured grids. The finite difference can be described as follows:

$$\begin{aligned} f_{xx} &= \frac{x_{i+1,j} - 2x_{ij} + x_{i-1,j}}{h^2} \\ f_{xy} &= \frac{x_{i+1,j+1} - x_{i-1,j-1} + x_{i-1,j+1} - x_{i+1,j-1}}{4h^2} \\ f_{yy} &= \frac{x_{i,j+1} - 2x_{ij} + x_{i,j-1}}{h^2} \end{aligned} \quad (2)$$

where h represents the step length, and x_{ij} represents the grid value at position (i, j) .

Constrained with a set of sampling points, Equation (1) can be further transferred into an iterative form (HASM) as follows:

$$FX^{(n+1)} = T^{(n)} \quad (3)$$

where F represents the coefficient matrix, X the unknowns, and T the right side item of Equation 1, and X^n and T^n represent the n th iteration result of X and T , respectively. It should be noticed that the coefficient matrix F , with a dimension of around $N \times N$, where N is the number of grid cells, is a rather huge matrix. Even when it is stored in a compressed form, with only the non-zero entries in a triple form stored, it still has to store elements with a number of nearly 9 times of N , which surely leads to a large consumption of memory (Yue, 2011).

For a large scale surface, this linear system is large due to numerous equations equaling the number of grid cells. Therefore, solving HASM is normally computationally intensive depending on the resolution and the study area.

2.2 | GPU and multi-grid

A GPU is increasingly applied in compute intensive applications due to its massive computational power from hundreds to thousands of cores and highly parallel architecture. The tasks on GPUs can be decomposed into numerous smaller tasks, which could be mapped to multiple threads run by such cores with a Single Instruction Multiple Data (SIMD) strategy. In this strategy, numerous threads execute the same instruction on the different data allocated to them, and executed in a highly parallel manner. The memory on a GPU comprises a hierarchy of three levels, global memory, shared memory and register. These three levels of memory have different capacities and access speeds and are designated for different tasks. The utilization of these memories should take into account their properties and make trade-offs between access speed and memory capacity.

Although numerous applications have demonstrated the potential of the GPU for delivering significant performance gains on computing intensive, large and complex problems (Owens et al., 2008), porting an algorithm from CPU to GPU is a nontrivial task due to the completely different architectures. The strategy to parallelize the task should be carefully designed, because the numerical method of the algorithm should be well chosen and devised, and the memory access should be dealt with carefully to gain an optimal performance. Additionally, to devise efficient mappings of existing algorithms to the streaming compute paradigm with thousands of lightweight hardware threads being executed simultaneously is a real challenge (Goddeke et al., 2008).

Multi-grid methods belong to the fastest and most versatile schemes available for numerical solution of linear systems (Briggs & McCormick, 2000). It can reach a linear complexity of computing time. The idea of Multi-grid is

motivated by the following error smoothing theory. General iterative methods such as Jacobi, Gauss-Seidel, etc. can eliminate high frequency components of the error effectively, while leaving the low frequency or smooth components relatively unchanged (Briggs & McCormick, 2000). It is found that when these smooth components are projected onto a relatively coarser grid, they will become more oscillatory, thus becoming easier to eliminate. According to this idea, a sequence of coarse grids could be created and employed to smooth out the errors. When the errors become low frequency errors and difficult to eliminate on the current grid, they should be projected onto the next coarser grid, on which the low frequency errors become the high frequency errors and are easy to eliminate. This process should be repeated until they reach the coarsest grid on which they could be directly solved with a short enough computing time.

Due to its high computing efficiency, multi-grid methods have widespread applications in numerical simulation, image processing and so on (McAdams, Sifakis, & Teran, 2010; Zhou, Huang, Xu, Guo, & Shum, 2007; Zhu, Sifakis, Teran, & Brandt, 2010). In recent years, with the development of parallel computing techniques, the parallel strategy was adopted to further improve the efficiency of multi-grid (Köstler, Feichtinger, Rude, & Aoki, 2014; McAdams et al., 2010). A well designed multi-grid method can be composed of completely parallel operations, and thus can benefit immensely from parallel architecture such as GPU (Grossauer & Thoman, 2008).

2.3 | Design

A typical multi-grid method complements several stages. Before the start, the initial value and the boundary conditions should be established, which involves Dirichlet and Neumann conditions, and a series of coarse grids should be generated. The first stage is an up-down procedure, involving smoothing out the high frequency error on the finest grid, calculating the residue between the exact solution and the approximate solution and projecting the calculated residue to the generated grid. These steps of the first stage should be repeated until it reaches the coarsest grid. The second stage involves solving the coarsest grid directly or iteratively. In the third stage, a down-up procedure is executed, which includes: prolonging the approximated solution to the finer grid and correcting the solution obtained last time; smoothing the approximated solution then repeating this process until it returns to the finest grid and obtains the final solution. If the error requirement cannot be satisfied, the aforementioned three stages should be repeatedly executed until the error can be decreased enough to satisfy the established error threshold. It should be noticed that the near boundaries grid should be treated with care due to the different number of grids involved when smoothing out the error.

The overall framework for a three level V-cycle multi-grid is described in Figure 1, where R0 and R1 represent the restriction operation to the next coarser level, P0 and P1 represent the interpolation operation to the finer level, and A0, A1 and A2 represent the coefficient matrix

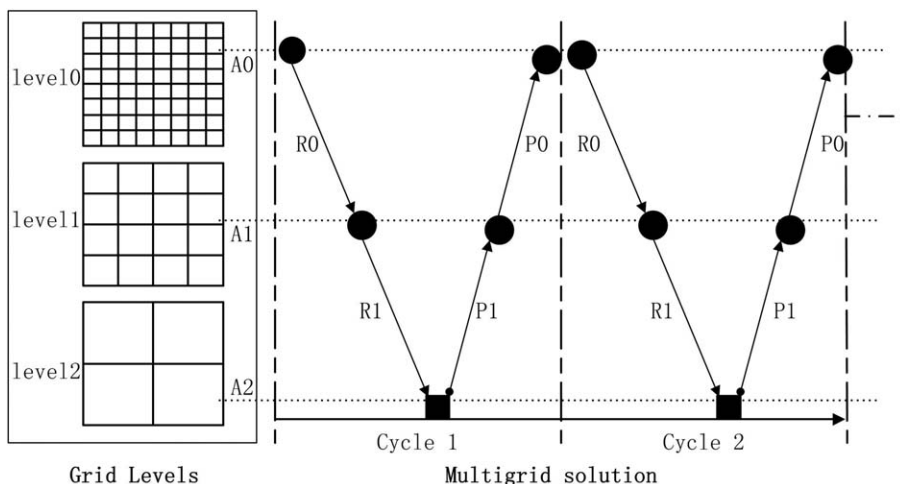


FIGURE 1 Multi-grid framework (R0 and R1 represent the restriction operation, P0 and P1 represent the interpolation operation, and A0, A1 and A2 represent the coefficient matrix)

A0, A1 and A2 represent the coefficient matrix of corresponding level. One V-cycle denotes a whole fine-coarse-fine process.

In terms of the multi-grid algorithm, a multi-grid method mainly involves five operations. The detailed design of each operation of our multi-grid can be presented as follows.

2.3.1 | Coarser grid generation

Coarser grid generation is essential for a multi-grid method. There are two commonly used methods, one is discretization coarse grid approximation (DCGA), and the other is Galerkin coarse grid approximation (GCA). DCGA generates a coarse grid through discretization of the solution domain according to the corresponding scale of cell unit in each level, and GCA calculates the coarse grid through an expression as follows:

$$A_{coar} = PA_{fine}S \quad (4)$$

where P denotes the matrix for projection, and S denotes the restriction matrix. Both methods have advantages and disadvantages. DCGA is advantageous for its simplicity, for not destroying the nonzero pattern of the original coefficient matrix A , and a relatively less memory consumption compared to GCA, while GCA is favored over DCGA because of its fast convergence rate, robustness and sound theoretical background. In this research, considering the large scale memory cost spent by HASM, we chose DCGA method.

2.3.2 | Relaxation method

For the smoothing stage, the damped Jacobi is a suitable method for its rapid high frequency error smoothing effect (Briggs & McCormick, 2000). Furthermore, it can be parallelized with less effort than other methods (Briggs & McCormick, 2000). The damped Jacobi can be expressed is follows:

$$u^{(n+1)} = u^{(n)} + \omega D^{-1} r^{(n)} \quad (5)$$

where u denotes the vector of unknowns, the superscript denotes the n th iteration, ω denotes the weight value ranging from 0 to 1, D denotes the diagonal of the coefficient matrix of the given linear system, and r denotes the residue vector between the nearest two results of calculated unknowns.

2.3.3 | Prolongation

For interpolation or prolongation, many interpolation methods can be used, of which the simplest linear interpolation is effective. Thus, we consider only the linear interpolation method. The interpolation can be represented as follows:

$$u_{2i,2j}^h = u_{ij}^{2h} \quad (6)$$

$$u_{2i+1,2j}^h = \frac{1}{2} (u_{ij}^{2h} + u_{i+1,j}^{2h}) \quad (7)$$

$$u_{2i,2j+1}^h = \frac{1}{2} (u_{ij}^{2h} + u_{i,j+1}^{2h}) \quad (8)$$

$$u_{2i+1,2j+1}^h = \frac{1}{4} (u_{ij}^{2h} + u_{i+1,j}^{2h} + u_{i,j+1}^{2h} + u_{i+1,j+1}^{2h}), \quad 0 \leq i, j \leq \frac{n}{2} - 1 \quad (9)$$

where h represents the grid size of the fine grid, $(2i, 2j), (2i, 2j+1), (2i+1, 2j)$ and $(2i+1, 2j+1)$ represent the neighboring grids on the fine-grid level, $(i, j), (i, j+1), (i+1, j)$ and $(i+1, j+1)$ represent the neighboring grids on the coarse-grid level, and n denotes the grid number of each line or column in the coarse-grid. These equations show how to obtain the grid values for the fine-grid level from the grid values of the coarse-grid level. They indicate that at even-numbered fine-grid points, the values are transferred directly from Ω^{2h} to Ω^h , while at odd-numbered ones, the values are the average of the adjacent coarse-grid values. We denote the interpolation operator (Equations 6–9) with I_{2h}^h , and the interpolation procedure can be formulated as:

$$u^h = I_{2h}^h u^{2h} \quad (10)$$

where u represents the vector of unknowns, u^h represents the vector of unknowns in the fine grid, and u^{2h} represents that in the coarse grid.

2.3.4 | Restriction

The restriction method denotes the fine to coarse operator, where nine neighboring points are employed to calculate the corresponding value of the coarser grid. These nine points are assigned different weights according to their distance from the point to be calculated. The restriction operator can be expressed as follows:

$$u^{2h} = I_h^{2h} u^h \quad (11)$$

In our research domain, we adopted the weighting restriction operator summarized below:

$$u_{ij}^{2h} = I_h^{2h} u^h = \frac{1}{16} \left[u_{2i-1,2j-1}^h + u_{2i-1,2j+1}^h + u_{2i+1,2j-1}^h + u_{2i+1,2j+1}^h + 2(u_{2i,2j-1}^h + u_{2i,2j+1}^h + u_{2i-1,2j}^h + u_{2i+1,2j}^h) + 4u_{2i,2j}^h \right] \quad (12)$$

where u_{ij}^h and u_{ij}^{2h} represents the unknown in position (i,j) in the fine grid and the coarse grid, respectively; and

$$I_h^{2h} = \begin{bmatrix} 1/16 & 1/8 & 1/16 & 1/8 & 1/4 & 1/8 & 1/16 & 1/8 & 1/16 \end{bmatrix} \quad (13)$$

2.3.5 | Correction

The correction operation involves computing a vector addition and can be expressed as follows:

$$u^h = u_{previous}^h + I_h^{2h} u^{2h} \quad (14)$$

where $u_{previous}^h$ represents the value stored during the fine to coarse process.

2.4 | Implementation

Combining HASM and the multi-grid method with the GPU technique, we develop HASM-GMG, which solves HASM with a GPU accelerated multi-grid. For performance comparison, HASM-MG, which solves HASM with multi-grid running on the CPU, was also implemented. Additionally, HASM with preconditioned conjugate gratitude (HASM-PCG) and, GPU accelerated HASM-PCG (HASM-GPCG) were also employed as references.

The HASM coefficient matrix has only nine nonzero entries at most in each line. These nonzero entries have fixed values and regular distributions in the whole matrix. In each line, each of these nine nonzero entries can be denoted by 1 for existing or 0 for not, with their relative positions unchanged. Consequently, these nonzero entries of each line can be coded as a series of 0s and 1s. Since the diagonal entry in each line always exists, we can use a variable with 8 bits to record other nonzero entries. Treating them in this way will save a huge amount of memory. Further, it will facilitate the parallel programming and improve its running speed.

According to this principle, the code denoting whether or not the corresponding elements exist should be calculated first. It will be employed by the relaxation and residue operations.

With the coefficient matrix presented by coded series of 0s and 1s, the parallelization of relaxation algorithm could be accomplished with ease and high efficiency and the program branch structure such as if-else statements, which are not well adapted to parallel architecture, were decreased significantly, and further improved the parallel program running speed on GPU. Additionally, it is worth noting that the border should be carefully treated for each level, or the singular value could be generated. Furthermore, the last thread block is possibly not large enough to constituent a standard block, which should be specially treated.

The residue operation can be implemented in a similar way as the relaxation operation. Compared with the two operations, other operations such as prolongation, restriction, coarse grid generation and correction, are relatively easier to implement. Since these operations can be implemented in a similar way, we do not show all the implementations.

3 | EXPERIMENTS AND RESULTS

To validate our method, two examples were designed and employed to test it. All the tests were run on a GeForce gt550m GPU with 96 CUDA cores hosted by an Intel core i5 2410 CPU in a notebook. It should be noticed that the experimental environment is configured with rather low-end hardware, especially for the GPU. In the experiments, we

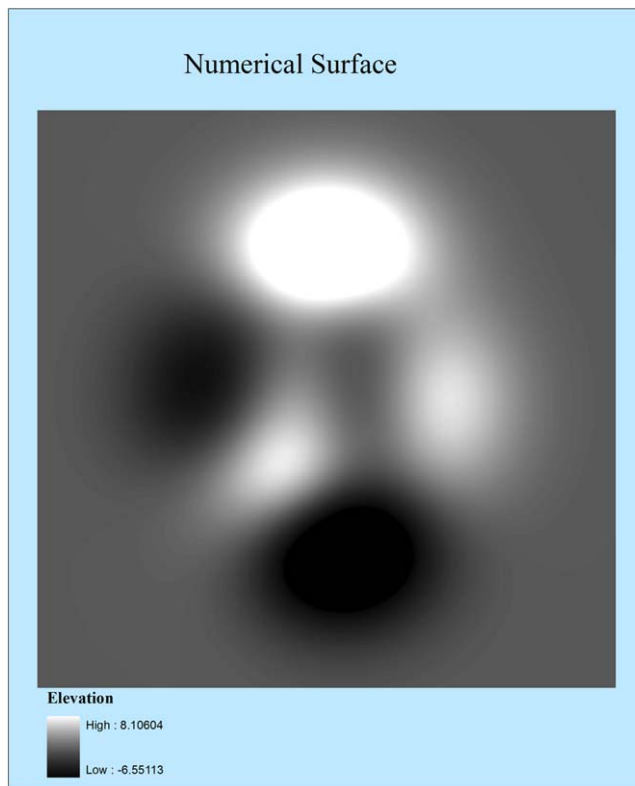


FIGURE 2 Numerical surface

employed the timer in C++ and the CUDA function to measure the running time for HASM-GMG and the timer in ArcGIS for Kriging, Spline and IDW.

The first example is a synthetic numerical surface (Figure 2), which can be expressed with the following equation:

$$f(x, y) = 3(1-x)^2 e^{-x^2 - (y+1)^2} - 10(x/5 - x^3 - y^5) e^{-x^2 - y^2} - e^{-(x+1)^2 - y^2} / 3 \quad (15)$$

where $f(x, y)$ represents the elevation value at the position (x, y) .

We chose this surface so that the true value of the surface could be predetermined for the error analysis. This surface has an elevation range between 6.05 and 8.0 m. Its x and y coordinates both range from -3 to 3 m. We approximate this surface employing five resolutions, which are 251 by 251 , 502 by 502 , $1,001$ by $1,001$, $2,002$ by $2,002$, and $4,004$ by $4,004$, respectively. The numbers of points random sampled in each resolution are 750 , $2,500$, $5,000$, $10,000$ and $15,000$, respectively.

Both HASM-GMG and HASM-MG were employed to model this surface. Before the surface modeling start, the necessary parameters should be established, such as the pre-smoothing iteration number, the post-smoothing number, and the grid level number. Generally, only one or two iterations are sufficient to smooth out the high frequency error (Hackbusch, 1985); therefore, we established the pre-smoothing iteration and post-smoothing numbers as 2 and 1 , respectively. The selection of the grid level number ensured that the coarsest grid level had a small enough number of

TABLE 1 The computing time comparison of HASM-MG and HASM-GMG on different scale numerical surface

Grid Number	251x251	502x502	1,001x1,001	2,002x2,002	4,004x4,004
HASM-MG (seconds)	1.402445	2.341330	4.732511	9.395020	18.697143
HASM-GMG (seconds)	0.079698	0.123157	0.247172	0.481051	1.054193
Ratio	17.597	19.011	19.147	19.530	17.702

TABLE 2 The minimum and maximum value of the residue in each multi-grid level for the Gauss synthetic surface

2002*2002	Level 0	Level 1	Level 2	Level 3	Level 4
Min (meters)	−0.002415	−0.000339	−0.000180	0.000000	0.000000
Max (meters)	0.002434	0.000362	0.000155	0.000000	0.000000

grids so that the linear system on it can be solved rapidly (Hackbusch, 1985). It was established as five in our experiments. Moreover, the stopping criteria that was set for solving Equation 3 in HASM-GMG and HASM-MG was $\|T-FX\|_2 \leq 10^{-9}$. Table 1 shows the computing time for five grid scales using HASM-MG and HASM-GMG when reaching the established stopping error.

To investigate the residual changes between different grid levels, the maximum and minimum error were recorded (Table 2).

Previous studies indicated that HASM-PCG has a superior convergence rate over most other iterative methods (Yue et al., 2013). Therefore we made a comparison on convergence rate between HASM-PCG, HASM-GPCG and HASM-GMG (Figure 3). It can be seen that HASM-GMG had a higher convergence rate for iterations at the beginning, about twenty for this example, and a decreasing convergence rate afterwards, with the stopping error almost remaining unchanged. In contrast, the stopping error of HASM-PCG maintained a slow but relatively stable decreasing trend, and it would surpass HASM-PCG after about 190 iterations. Additionally, HASM-GPCG had a similar trend as HASM-PCG but an obviously higher speed than HASM-PCG.

The second test surface is a real-world DEM (Figure 4), which is the Poyang lake basin in Jiangxi province. We chose it to test HASM-GMG for practical use. Its elevation ranges from 30 to 1,580 m. We chose three regions, denoted as DEM A, B and C, the grid scales of which are 2,587 by 3,041, 2,587 by 3,041, and 2,562 by 2,907, respectively from Poyang lake basin. The cell size of these three regions is 40 m. Poyang lake basin has a complex topography, with mountainous and hilly areas, which is appropriate for DEM test. The data source was digitized contours from a map, which was first discretized into scattered points set with ArcGIS tool feature vertices to points module. The number of points used was 209,458 in region A, 96,154 in region B, and 203,832 in region C. In every region, the points from it were divided into two parts with 90% used as constraint sampling points to drive the surface modeling process, and 10% used for accuracy validation. The HASM-MG, HASM-GMG, HASM-PCG and HASM-GPCG methods were employed to model these DEM surfaces. DEM A was used to test the convergence rate and computing speed of these methods and all three regions were used for accuracy comparison between HASM-MG, HASM-GMG and the classical methods of IDW, Spline, Kriging.

To further investigate the residual changes for real-world DEM surface, the residual changes between different grid levels were also recorded (Table 3).

For the real-world examples, the errors showed decreasing trends for HASM-GMG, HASM-MG and HASM-PCG (Figure 5). From the start to a certain point, HASM-GMG and HASM-PCG had a similar error decreasing speed. After

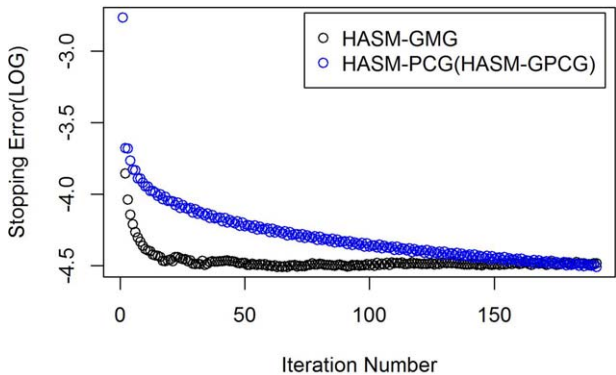


FIGURE 3 Comparison between HASM-GMG, HASM-PCG and HASM-GPCG for the Gauss synthetic surface modeling

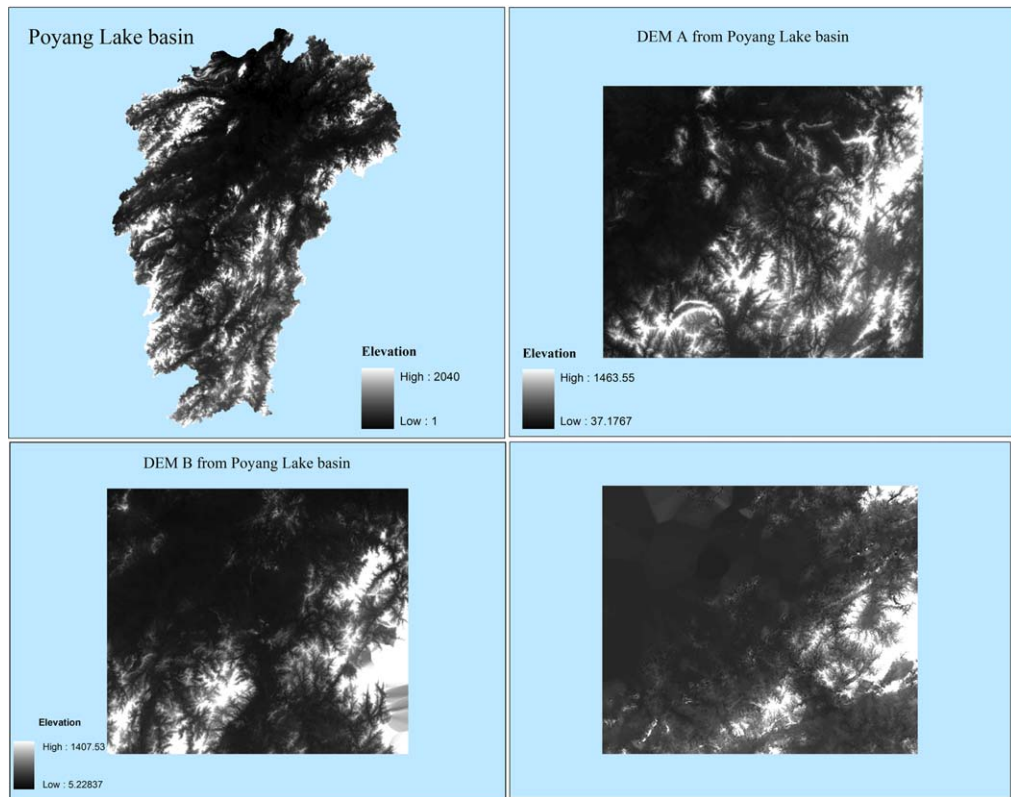


FIGURE 4 Real-world DEM surface

that point, the error decreasing speed of HASM-GMG was lower than that of HASM-PCG. The error decreasing speed differed between HASM-PCG and HASM-GMG, which first increased to a maximum, and then decreased with the computing time. After a sufficient long computing time, the error decreasing speed of HASM-PCG and HASM-GMG converged and was very similar.

To evaluate the acceleration and accuracy exhibited by HASM-GMG, Kriging, IDW, and Spline were also employed to model these three surfaces. For Kriging, the ordinary method and spherical models were employed, and the number of closest samples was 12. For IDW, the power was established as 2. For Spline, the regularized methods were employed using the same number of neighbors as that employed in OK and IDW. The accuracy was measured with the root mean square error (RMSE), which can be described as follows:

$$RMSE = \sqrt{\frac{\sum_{k=1}^N (h_k - \bar{h}_k)^2}{N}} \quad (16)$$

where h_k is the true value at the k th position (x_i, y_j) , $\bar{h}_k$ is the simulated value, and N is the number of points for validation. According to the $RMSE$ criteria, the result with the smaller $RMSE$ should be more accurate, and the ideal accurate result should have the $RMSE$ equal or near to zero. The max, min, and average error are also reported. Tables 4, 5, and

TABLE 3 The minimum and maximum value of the residue in each multi-grid level for the real-world surface

2176* 2370	Level 0	Level 1	Level 2	Level 3	Level 4
Min (m)	−137.452637	−22.194036	−5.552900	0.000000	0.000000
Max (m)	119.011871	26.425495	5.786547	0.000000	0.000000

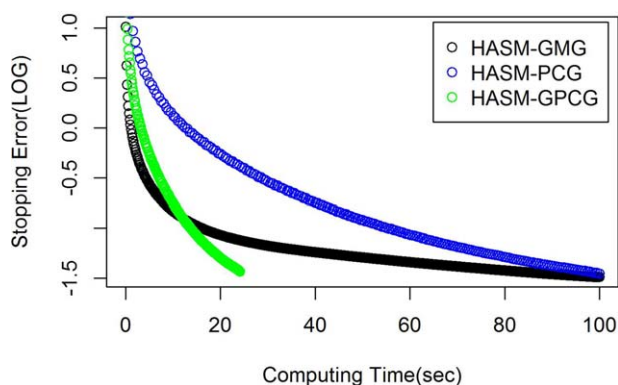


FIGURE 5 Comparison between HASM-GMG, HASM-PCG and HASM-PGCG for the real-world DEM surface modeling

6 show the accuracy and computing time of the three classical methods, HASM-MG and HASM-GMG for the three regions labeled A, B and C in Figure 4.

For all the methods, HASM-MG has the biggest computing time and HASM-GMG the smallest. In terms of accuracy, HASM-GMG and HASM-MG surpassed all the other methods. HASM-GMG outperformed all other methods in both accuracy and speed.

TABLE 4 Accuracy and computing time comparison for DEM A (X represents the same result as HASM-GMG)

2,587*3,041	IDW	Kriging	Spline	HASM-GMG	HASM-MG
TIME(sec)	233.1	533.5	238.2	103.3	3001.6
Ratio	2.256534	5.164569	2.305905	1	29.05712
RMSE(m)	20.06558	19.8384	22.51509	6.87882	X
AVG(m)	12.78043	13.49708	14.38234	3.844928	X
MAX(m)	190.3586	146.989	187.3925	107.0428	X
MIN(m)	0.00056	0.00028	0.00031	0.00001	X

TABLE 5 Accuracy and computing time comparison for DEM B (X represents the same result as HASM-GMG)

2,587*3,041	IDW	Kriging	Spline	HASM-GMG	HASM-MG
TIME(sec)	561	2241	568	104.5	3146.9
RATIO	5.368421	21.44498	5.435407	1	30.11388
RMSE(m)	30.62057	33.78286	45.4346	9.688272	X
AVG(m)	21.36758	24.68054	30.34718	5.948848	X
MAX(m)	255.5471	256.1498	463.898	159.944	X
MIN(m)	0.000144	0.000123	0.000119	0.000038	X

TABLE 6 Accuracy and computing time comparison for DEM C (X represents the same result as HASM-GMG)

2,562*2,907	IDW	Kriging	Spline	HASM-GMG	HASM-MG
TIME(sec)	599.3	2049.1	592.2	98.9	2931.9
RATIO	6.059656	20.71891	5.987867	1	29.6451
RMSE(m)	32.2328	33.53833	46.72149	10.10345	X
AVG(m)	21.89852	24.00135	32.56506	6.140031	X
MAX(m)	243.3328	261.2485	874.6775	172.6011	X
MIN(m)	0.000122	0.000106	0.000123	0.0001	X

TABLE 7 Computing time for large DEMs modeling

Grid Number	IDW (sec)	Kriging (sec)	Spline (sec)	HASM-GMG (sec)	HASM-MG (sec)
5,812*5123	620.3	3,603.5	871	550	6,696
6,081*5,173	691.3	3,526.1	868.2	620.9	7,506.8
11,624*10,246	937	8,042	1,868	overflow	overflow
12,160*10,346	1,000.9	7,226	1,846.8	overflow	overflow

To further test the computing speed for large DEMs, we constructed four DEM surfaces with the resolution of 6,081 by 5,173, 5,812 by 5,123, 12,160 by 10,346 and 11,624 by 10,246, by changing the grid cell size of area A and B to 20 and 10 m, respectively. We carried out the surface modeling experiment to test the performance on a relatively large DEM. The results are showed in Table 7. It is found that when modeling a large DEM with the number over 10,000 by 10,000, the 4G main memory configured in our computer was not sufficient and caused an overflow error.

4 | DISCUSSION

With the implementation of the multi-grid method, it significantly reduced the computing time for surface modeling without compromising the modeling accuracy. For the first time, the multi-grid method is used to solve the linear-systems of HASM on a GPU. This method proved to be suitable to take the advantage of the parallelism of a GPU.

First, residual changes across different levels are not consistent. Both the numerical surface (Table 2) and the real-world example (Table 3) indicates that four levels is enough for a multi-grid method to achieve the highest benefit-to-cost ratio in terms of residual reduction due to the zero residual at levels 4 and 5 in our tests. Thus, we could conclude that the multi-grid with four or five levels is appropriate for our surface modeling if we pursue only the reduction of residuals. Nevertheless, more levels could contribute to the computing time reduction and help the multi-grid method gain linear time complexity due to the smaller scale of the coarsest grid, which generally is solved by direct methods (Briggs & McCormick, 2000; Hackbusch, 1985). Since we focus on acceleration of the multi-grid with the utilization of GPU computing, this is beyond the scope of this study.

Second, the convergence of HASM-MG, HASM-GMG and HASM-PCG changes with the increasing time or number of iterations. From Figures 3 and 5, we found that the stopping error of HASM-MG or HASM-GMG could be reduced rapidly for the former iterations or time period, and then it decreased very slowly and perhaps fluctuated, compared with HASM-PCG. The smallest number of iterations to match the least stopping error is about 20 for our example (Figure 3). Consequently, the multi-grid should be executed for the number of iterations that produces the best performance/cost ratio. HASM-GMG can gain around 20 times speedup against HASM-MG for simulation of a numerical surface (Table 1) and 30 times against HASM-MG for a real-world surface (Tables 4-6). Therefore, HASM-GMG is approximately more than 20 times faster to finish the same iterations or reach the same stopping error. HASM-PCG can gradually decrease the stopping error and maintain a decreasing trend, and at the same time HASM-PCG would attain higher accuracy after about 160 iterations. The superior speed could still maintain HASM-GMG in a leading position against HASM-PCG.

During a relatively short time period, HASM-GMG has a rapid convergence rate and its convergence rate outperforms HASM-PCG (Figure 5) but its convergence rate will become slow and will be eventually surpassed by HASM-PCG in a sufficiently long time if the execution is continued. This conforms to the findings in Yue et al. (2013) and can be attributed to the fact that the relatively high frequency errors at each level are completely smoothed out after several iterations, but those relatively low frequency errors are retained at each level. Nevertheless, since HASM-GMG has a much higher computing speed, only when required to match a rigorous stopping error threshold, could HASM-PCG have the opportunity to surpass HASM-GMG.

Third, HASM-GMG has a higher speed and accuracy than other classical methods. HASM-GMG has an approximate 30 times speedup against HASM-MG, and is faster than all the classical methods (Tables 4-6). Furthermore, its accuracy is higher than the classical methods, notably higher than Kriging, moderately higher than Spline, and slightly

higher than IDW. However, it should be noted that the parameters of all the methods, including HASM-GMG, Kriging, Spline and IDW, were not well optimized, which will be our next research focus.

Additionally, it should be noted that HASM-GMG has not addressed the large scale surface modeling problem (Table 7), since large scale surface modeling involves large scale spatial data processing and requires large main memory to accommodate the data, which could be addressed on a distributed processing platform such as a MapReduce model (McDermott, Prasad, Shekhar, & Zhou, 2015). A MapReduce model can partition a huge computing task into multi smaller task and assign each of them to a computer node, enable multi hosts and GPUs run in parallel and finish these small tasks at the same time (Dean & Ghemawat, 2008). Porting HASM-GMG onto such a platform can address large-scale surface modeling problems.

5 | CONCLUSIONS

To summarize, we present a GPU-accelerated method which is able to accelerate the solving of the HASM linear equations for surface modeling, HASM-GMG. The presented method adopts the multi-grid algorithm and fully utilizes the GPU high performance computing power. Our experiments showed that HASM-GMG outperforms other methods with regard to both accuracy and speed. Compared with the CPU version HASM-MG, HASM-GMG attains 20 to 30x speedups. Although it will be surpassed by the HASM-PCG to satisfy an extremely low stopping error, HASM-GMG may significantly widen HASM applications due to its high speed and accuracy. Additionally, it should be noted that, equipped with a higher-end GPU, the speed of HASM-GMG can be expected to be further improved. Furthermore, the emerging MapReduce framework can partition a large-scale surface modeling task into numerous smaller tasks, and accomplish them simultaneously, which can fundamentally solve large scale surface modeling problems. That is also our future research.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (Grant Nos. 41101433, 41371367), by National Statistics Research Project (Grant No. 2013LZ16), by the Natural Science Foundation of Shandong Province (Grant No. ZR2013DM011), by Shandong University of Science and Technology Research Fund, by the Joint Innovative Center for Safe and Effective Mining Technology and Equipment of Coal Resources, by the State Key Laboratory of Resources and Environmental Information System and by the Start-up Foundation for Doctors of Shandong University of Science and Technology (Grant No. 2014RCJJ038).

REFERENCES

- Beutel, A., Mülhave, T., & Agarwal, P. K. (2010). *Natural neighbor interpolation based grid DEM construction using a GPU* (pp. 172–181). Paper presented at the eighteenth ACM SIGSPATIAL international conference on advances in geographic information systems, San Jose, California.
- Briggs, W. L., & McCormick, S. F. (2000). *A multigrid tutorial*. Philadelphia, PA, Society for Industrial Mathematics.
- Bryan, B. A. (2013). High-performance computing tools for the integrated assessment and modelling of social-ecological systems. *Environmental Modelling and Software* 39, 253–303.
- Chao, F., Chongjun, Y., Zhuo, C., Xiaojing, Y., & Hantao, G. (2011). Parallel algorithm for viewshed analysis on a modern GPU. *International Journal of Digital Earth* 4, 471–486.
- Childs, C. (2004). *Interpolating surfaces in ArcGIS spatial analyst*. *ArcUser*, 7, 3, 32–35.
- de Ravé, E. G., Jiménez-Hornero, F. J., Ariza-Villaverde, A. B., & Gómez-López, J. (2014). Using general-purpose computing on graphics processing units (GPGPU) to accelerate the ordinary kriging algorithm. *Computers and Geosciences* 64, 1–6.
- Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM* 51, 107–113.
- Feng, W., Gang, W., Deji, P., Yuan, L., Liuzhong, Y., & Hongbo, W. (2015). A parallel algorithm for viewshed analysis in three-dimensional Digital Earth. *Computers and Geosciences* 75, 57–65.
- Fisher, P. F., & Tate, N. J. (2006). Causes and consequences of error in digital elevation models. *Progress in Physical Geography* 30, 467–489.

- Goddeke, D., Strzodka, R., Mohd-Yusof, J., McCormick, P., Wobker, H., Becker, C., & Turek, S. (2008). Using GPUs to improve multigrid solver performance on a cluster. *International Journal of Computational Science and Engineering*, 4, 36–55.
- Grossauer, H., & Thoman, P. (2008). GPU-based multigrid: Real-time performance in high resolution non-linear image processing. In A. Gasteratos, M. Vincze, & J. K. Tsotsos (Eds.), *Computer vision systems* (Vol. 5008, pp. 141–150). Berlin, Germany, Springer Lecture Notes in Computer Science.
- Hackbusch, W. (1985). *Multi-grid methods and applications*. Berlin, Germany, Springer-Verlag.
- Helpenstein, R., & Koko, J. (2012). Parallel preconditioned conjugate gradient algorithm on GPU. *Journal of Computational and Applied Mathematics* 236, 3584–3590.
- Köstler, H., Feichtinger, C., Rüde, U., & Aoki, T. (2014). A geometric multigrid solver on Tsubame 2.0. In A. Bruhn, T. Pock, & X. C. Tai (Eds.), *Efficient algorithms for global optimization methods in computer vision* (pp. 155–173). Berlin, Germany, Springer.
- McAdams, A., Sifakis, E., & Teran, J. (2010). A parallel multigrid Poisson solver for fluids simulation on large grids (pp. 65–74). Paper presented at the 2010 ACM SIGGRAPH/Eurographics symposium on computer animation, Madrid, Spain.
- McDermott, M., Prasad, S. K., Shekhar, S., & Zhou, X. (2015). Interesting spatio-temporal region discovery computations over GPU and MapReduce platforms. *ISPRS Annals of Photogrammetry, Remote Sensing and Spatial Information Sciences* 1, 35–41.
- McKenney, M., De Luna, G., Hill, S., & Lowell, L. (2011). *Geospatial overlay computation on the GPU* (pp. 473–476). Paper presented at the nineteenth ACM SIGSPATIAL international conference on advances in geographic information systems, Chicago, Illinois.
- Ortega, L., & Rueda, A. (2010). Parallel drainage network computation on CUDA. *Computers and Geosciences* 36, 171–178.
- Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., & Phillips, J. C. (2008). GPU computing. *Proceedings of the IEEE* 96, 879–899.
- Pan, J., & Manocha, D. (2011). *Fast GPU-based locality sensitive hashing for k-nearest neighbor computation* (pp. 211–220). Paper presented at the nineteenth ACM SIGSPATIAL international conference on advances in geographic information systems, Chicago, Illinois.
- Qin, C., & Zhan, L. (2012). Parallelizing flow-accumulation calculations on graphics processing units: From iterative DEM preprocessing algorithm to recursive multiple-flow-direction algorithm. *Computers and Geosciences* 43, 7–16.
- Stojanovic, N., & Stojanovic, D. (2013). High-performance computing in GIS: Techniques and applications. *International Journal of Reasoning-based Intelligent Systems* 5, 42–49.
- Tabik, S., Villegas, A., Zapata, E. L., & Romero, L. F. (2012). A fast GIS-tool to compute the maximum solar energy on very large terrains. *Procedia Computer Science* 9, 364–372.
- Yue, T. X. (2011). *Surface modeling: High accuracy and high speed methods*. New York, NY, CRC Press.
- Yue, T. X., Du, Z. P., Song, D. J., & Gong, Y. (2007). A new method of surface modeling and its application to DEM construction. *Geomorphology* 91, 161–172.
- Yue, T. X., Zhao, N., Yang, H., Song, Y. J., Du, Z. P., Fan, Z. M., & Song, D. J. (2013). A multi-grid method of high accuracy surface modeling and its validation. *Transactions in GIS* 17, 943–952.
- Zhang, J., & You, S. (2012). *CudaGIS: Report on the design and realization of a massive data parallel GIS on GPUs* (pp. 101–108). Paper presented at the third ACM SIGSPATIAL international workshop on geostreaming, Redondo Beach, California.
- Zhao, G., Bryan, B. A., King, D., Luo, Z., Wang, E., Bende-Michl, U., Song, X., & Yu, Q. (2013). Large-scale, high-resolution agricultural systems modeling using a hybrid approach combining grid computing and parallel processing. *Environmental Modelling and Software* 41, 231–238.
- Zhao, G., Bryan, B. A., King, D., Song, X., & Yu, Q. (2012). Parallelization and optimization of spatial analysis for large scale environmental model data assembly. *Computers and Electronics in Agriculture* 89, 94–99.
- Zhao, Y., Cheng, Z., Dong, H., Fang, J., & Li, L. (2011). *Fast map projection on CUDA* (pp. 4066–4069). Paper presented at the IEEE international geoscience and remote sensing symposium (IGARSS), Vancouver, British Columbia, Canada.
- Zhao, Y., Padmanabhan, A., & Wang, S. (2013). A parallel computing approach to viewshed analysis of large terrain data using graphics processing units. *International Journal of Geographical Information Science* 27, 363–384.
- Zhou, K., Huang, X., Xu, W. W., Guo, B., & Shum, H. Y. (2007). Direct manipulation of subdivision surfaces on GPUs. *ACM Transactions on Graphics* 26, 911–918.
- Zhu, Y., Sifakis, E., Teran, J., & Brandt, A. (2010). An efficient multigrid method for the simulation of high-resolution elastic solids. *ACM Transactions on Graphics* 29, 1–16.